

AMBA AHB PROTOCOL : Tri-Slave Interconnect Design

NAME: Pratyush Ojha

INSTITUTE: Motilal Nehru National Institute of Technology Allahabad

GOAL: To increase performance by reducing extra clock cycle delay after every burst and minimize memory wastage.

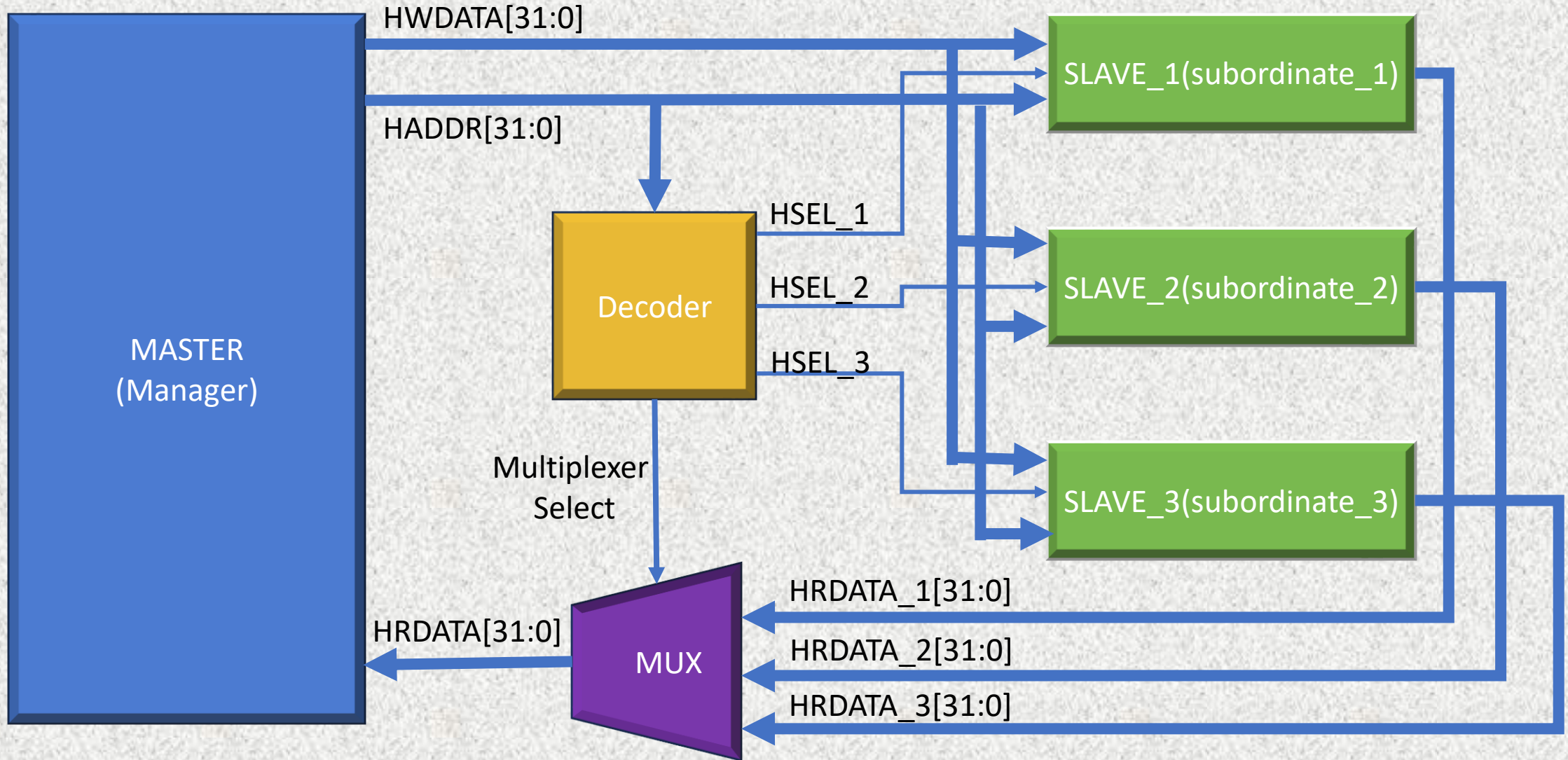
Context: SoC designs require standardized communication as complexity grows.

- In the early days of electronics, chips were very simple. You had a processor and a memory, and you just connected them together. It was like teaching a single student—you just look at them and talk.
- But modern SoCs contain a CPU, a GPU, a USB controller, RAM, and hardware accelerators (hardware accelerator is a dedicated block of silicon where the physical logic gates (AND, OR, XOR) are hardwired to solve one specific mathematical algorithm perfectly) all crammed onto a single piece of silicon. Now, you aren't tutoring one student; you are managing an entire lecture hall of 300 students. If everyone tries to talk at once, or if you don't have a strict set of rules for how to raise hands and take turns, the entire system collapses into chaos. As the chip gets more complex, you absolutely must have a standardized rulebook for communication.
- An AHB connection isn't just one wire. It is a 32-bit Address bus, a 32-bit Write Data bus, a 32-bit Read Data bus, plus control signals. That is roughly 100 physical microscopic copper wires. If the Master hardwires to 3 slaves, the Master suddenly needs **300 physical pins** just to connect to them.

The Solution: AMBA AHB centralizes the routing using a master-slave contract, decoders, and multiplexers.

- **The Master's Pin Count Drops:** Master is the main module which access data from other small modules(Slaves) which have purpose to collect data and store them till they have power supply. The Master needs one set of approx 100 pins because it get access of slaves with help of decoder and multiplexer.
- **The Decoder :** When the Master sends out an Address (e.g. 128), the Decoder acts as the post office. It reads the address, realizes it belongs to Slave 2, and flips the HSEL_2 switch to wake up only Slave 2.
- **The Multiplexer:** When the Slaves send their data back, the MUX ensures that only Slave 2's data is allowed to enter the Master's receiving lane. It physically blocks the other slaves from causing a collision.
- **The Contract:** The protocol provides strict rules (like the NONSEQ -> SEQ bursts and pipeline delays you coded) so that every module knows exactly when to read and when to write.

Tri-Slave Architecture



The Master (The Initiator)

- **Role:** Generates addresses, selects burst types (SINGLE, INCR, WRAP, BUSY), and manages the control signals (HTRANS, HSIZE).
- The HADDR is address for slave but whatever operation master do whether it is read or write, the HADDR is completely independent of master's internal memory, thus master has its own choice that where from its memory it access data for write operation and where from its memory it should select address for read operation, in general read or write from its own memory goes in increasing address fashion starting from 0 initial address but user can change according to its need.
- HADDR is of 32 bit.
- Master read data through read data bus called HRDATA which is also of 32 bit here, correspondingly master write data through write data bus called HWDATA which is also of 32 bit here.
- In timing diagram, HRDATA or HWDATA are always 1 cycle delayed respect to HADDR.
- Currently for operations we take negedge of clock or negedge of HCLK, but its your choice whether you want to take negedge or posedge.

Master input pins:

- HRESET(reset pin)
- HCLK(clock)
- HREADY
- HRDATA [31:0],
- data_out[31:0]
- add[31:0]
- add_m [31:0]
- HWRITE
- size [2:0]
- burst_type [2:0]
- trans[1:0]
- wrap_input [31:0]
- again
- new
- single_input [31:0]

Mater output pins:

- HADDR[31:0]
- HADDR_M[31:0]
- HWDATA[31:0]
- data_in[31:0]
- HSIZE[7:0]
- HBURST[2:0]
- HTRANS[1:0]
- N[7:0]
- M[7:0]
- O[7:0]
- count_no[31:0]
- count[31:0]
- burst_done

Slave input pins(all 3 slaves included):

- HWRITE
- HCLK
- HREADY
- HTRANS [1:0]
- HADDR [31:0]
- HWDATA [31:0]
- HSEL_1
- HSEL_2
- HSEL_3

Slave output pins(all 3 slaves included):

- HRDATA_1 [31:0]
- HRDATA_2 [31:0]
- HRDATA_3 [31:0]

Scaling of decoder:

- Decoder takes address given by master and decode it to give signal to the slave for which master want to do read or write operation.
- Decoder decides and gives signals HSEL_1, HSEL_2 and HSEL_3.
- Slave 1: Addresses 0 to 124 → HSEL_1 = 1 and others 0
Slave 2: Addresses 128 to 252 → HSEL_2 = 1 and others 0
Slave 3: Addresses 256 to 380 → HSEL_3 = 1 and others 0
- Decoder has one more signal of 2 bit called Multiplexer Select or m_select, it has select lines which are given in mux which decides that from which slave it has to take data and give to master.
- If HSEL_1 == 1 corresponding m_select = 0, thus mux takes HRDATA_1 or takes data from slave 1.
- If HSEL_2 == 1 corresponding m_select = 1, thus mux takes HRDATA_2 or takes data from slave 2.
- If HSEL_3 == 1 corresponding m_select = 2, thus mux takes HRDATA_3 or takes data from slave 3.

burst_type:

- There are total 8 burst_type, whenever we are going to do any burst then it should be done in one slave at a time, for example: if burst_type = 3'b110, then it has total of 16 addresses for which we do read or write operation of 16 set of data, basically combine of them called 16 beats. It should not be the case that half of beat completed in one slave and other half completed in other slave, if this happen then it is protocol violation.
- In this way we can say that the initial_address(add or wrap_input) is chosen such that HADDR do not cross slave boundary.
- Burst are of two types:
 1. Incrementing burst: we give initial_address(add) and burst_type and it goes increasing till beats get over.
 2. Wrapping burst: we give initial_address(wrap_input), add and burst_type and address goes on increasing till it reaches Last_address of that particular burst and wrap to add and again goes on increasing till wrap_input - 4 or address just before wrap_input.
- $\text{Last_address} = \text{add} + ((\text{beat}-1) \times 4)$

burst_type:

- Here we are not wasting a single cycle unless if after any given burst_type we give any other burst_type but after a delay of 2 or more than 2 cycles, if this case happen then burst_done becomes 1.
- Hence to start the new burst instantly after old burst we need to give burst_type and initial_address just after the negedge of clock where we are getting last address of previous burst, otherwise burst_done becomes 1 and after that if we want to start new burst then we need to give trans = 2'b00 in testbench so that burst_done becomes 0 and then we can give new burst_type and initial_address, but in this situation it wastes 2 extra cycles.
- Only in burst_type = 3'b001 we do not have burst_done logic, because it depends on user when he/she want to complete the burst, hence when user want to complete the burst then just need to give trans = 2'b00.
- Here burst_done is a necessary part because if we not require any new burst_type then previous burst not repeats it self and get saturated after beat completion.
- Hence increasing data processing speed is a crucial step, so we required to give burst_type and initial_addresses at the moment when previous burst last address is given from master, this insures that we not waste any single clock cycle.

burst_type:

- In this situation we get a case that if we want previous burst to happen again just after that previous burst only then we do what?
- For this we have a input signal called “again”, this input signal uses only when we want same burst without any single clock cycle wastage.
- Assume you not give again = 1, then in this situation burst get saturated(burst_done = 1) and you need to give trans = 2'b00 so that burst_done = 0, hence this takes 2 extra cycles and it makes speed of protocol slower, remember that after every long time when you give new burst_type then it takes 2 extra cycles to start.
- Why it takes 2 extra cycle?

Remember given input in testbench if it is synchronized with HCLK comes in action after next negedge of HCLK only, so when if you not give any new burst_type and initial_address then burst_done becomes 1 hence to start again we need to give trans = 2'b00 so that burst_done becomes 0 as discussed above, so 1 cycle of burst done = 1 and other cycle of burst_done = 0 and when burst_done = 0 it comes only in action after next negedge of HCLK, in this situation it takes one more cycle and total of 2 cycles are taken to come in action.

burst_type:

Below I discussed few limitations of this RTL design burst logic:

- For starting a new burst, the initial counter value for that burst is set 0, when it starts counting, counter value starts increasing and increases till last beat value, and then becomes 0 again, here my main purpose is to increase speed or not waste any single cycle of HCLK unless burst_done = 1, I use different counters for different burst_type, you can use single counter too but for that when your counter value = last beat – 1, then you need to make it 0 and when next negedge of HCLK comes then it starts again counting from 1, for this you need to change logics of wrapping and incrementing in my code, previously when I am working on this and got stuck then I just thought for different counter, it has only limitation that number of flip flops and multiplexers increases but increasing few hundred of more digital devices is not a big issue of occupying more space in single chip.
- Here the best way to code for different burst_type is to use case statements, I use multiple always blocks, which in turn increases code length, later on when I thought of it then I just thought that okk I am just using more code space.
- If you give new burst_type in between previous burst is going now then it completely starts with that new burst_type. I thought that burst_type is user defined currently and HBURST is forming according to that, so whenever user need instant new burst data between any burst so he/she just need to give new burst_type.

I will remove these limitations soon.

HBURST and HTRANS:

- Whenever new burst starts then whatever the burst_type is given, HBURST set according to that.
- When new HBRUST occur then first HADDR is non sequential and then it becomes sequential. So HTRANS = 2'b10 in starting of burst then becomes 2'b11 till last beat.
- When master is in IDLE(2'b00) condition, means no any read and write operation, hence master do no give or take data.
- There is one more signal of HTRANS = 2'b01 which is BUSY state, master just give that signal between any burst when it stuck at any other task, in that case too master not take or give any data.

Main difference between IDLE and BUSY:

- IDLE (2'b00) is hanging up the phone. The Master is saying "I have no more data for you. Our conversation is over." If the Master wants to talk to that Slave again, it has to dial the number again (send a brand new NONSEQ command).
- BUSY (2'b01) is putting the Slave on hold. The Master is saying "We are in the middle of a burst, and I am not done talking to you. But my internal brain needs a second to fetch the next word. DO NOT hang up, I will be right back".

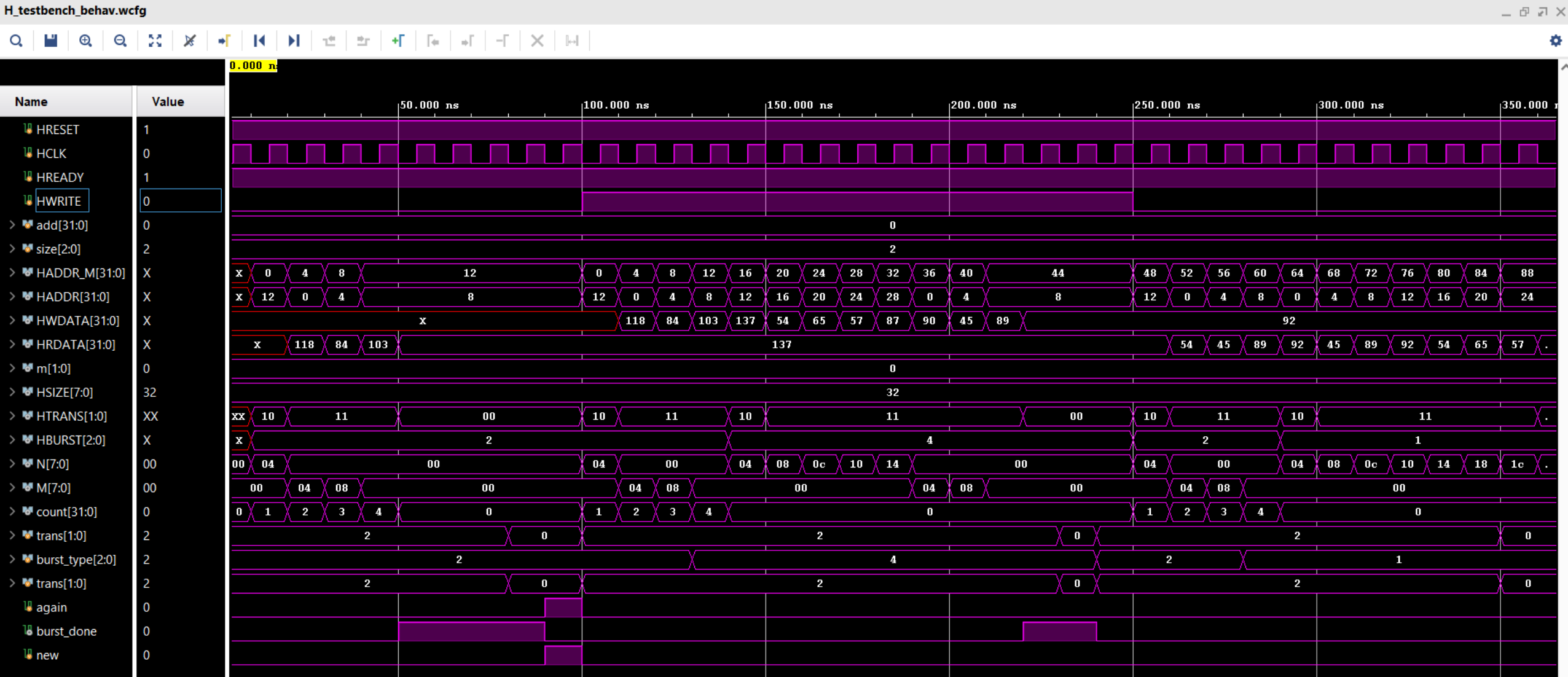
The HADDR_M logic:

- Master is independent candidate in reading and writing data, it is the choice of master where it want to store data in its own memory and where it from where I want to give data from its own memory too.
- In general it stores and give data sequentially as address increases, we give initial address of master memory as `add_m` which is input and given in testbench. HADDR_M is synchronized with HADDR, see whatever the burst it's the choice of master to give it according to initial address or `wrap_input`. If we constantly increase HADDR_M with every negedge of HCLK then it wastes too much to memory in case when we are in IDLE condition, hence we need that HADDR_M must be synchronized with HADDR hence no memory wastage happens.
- What if you want to start from any different HADDR_M between data transfer, as we start with 0 initial address but what if you want to starts with new initial address which again be 0 or any other value, for this we have 'new' input signal, which when given 1 along with `add_m` which decides new initial address and then starts increasing, remember that once you make `new = 1` then after next clock make `new = 0` otherwise it continuously starts from `add_m` and change only `add_m` and not increases.

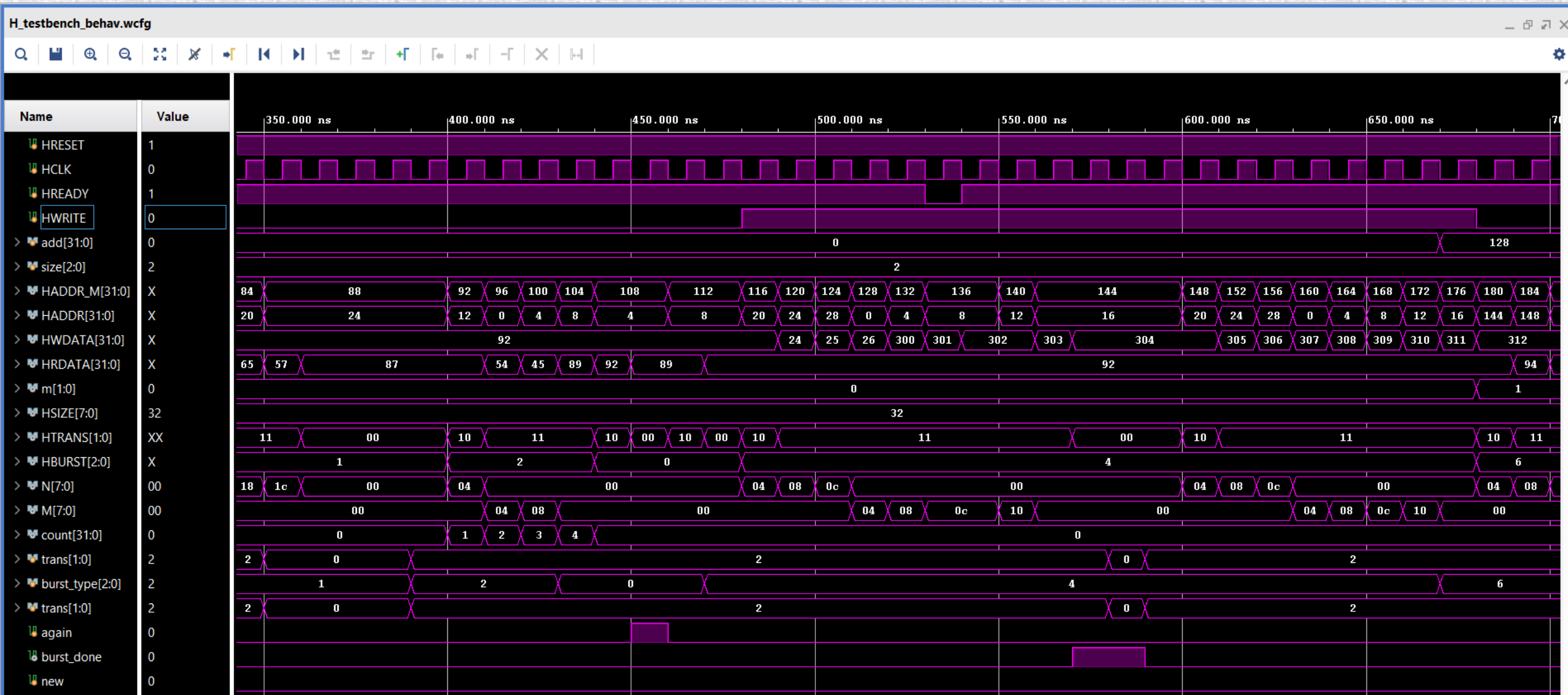
Pipelining the MUX :

- When we are about to change slave and give new initial addresses- add and wrap_input(for wrapping burst) then last address value of previous burst of previous slave is not loaded because m_select is level triggered and instantly when m_select changes then mux switches to new HRDATA and last data completely get losts.
- To avoid this problem we just need to think that whatever m_select we are giving to mux is same as the m_select coming from decoder and it is obvious, but if it get delayed by one cycle then whenever m_select coming from decoder changes, then the input m_select for mux changes one HCLK cycle later, hence last address value of previous burst of previous slave is loaded at the starting of new burst_type and then other data of new burst_type starts loading, hence in this scenario we do not waste any cycle and jump to other slave. For this we need to give a new register m_delayed in top module and code it such that it get its value after negedge of HCLK from m_select, hence get one cycle delayed.

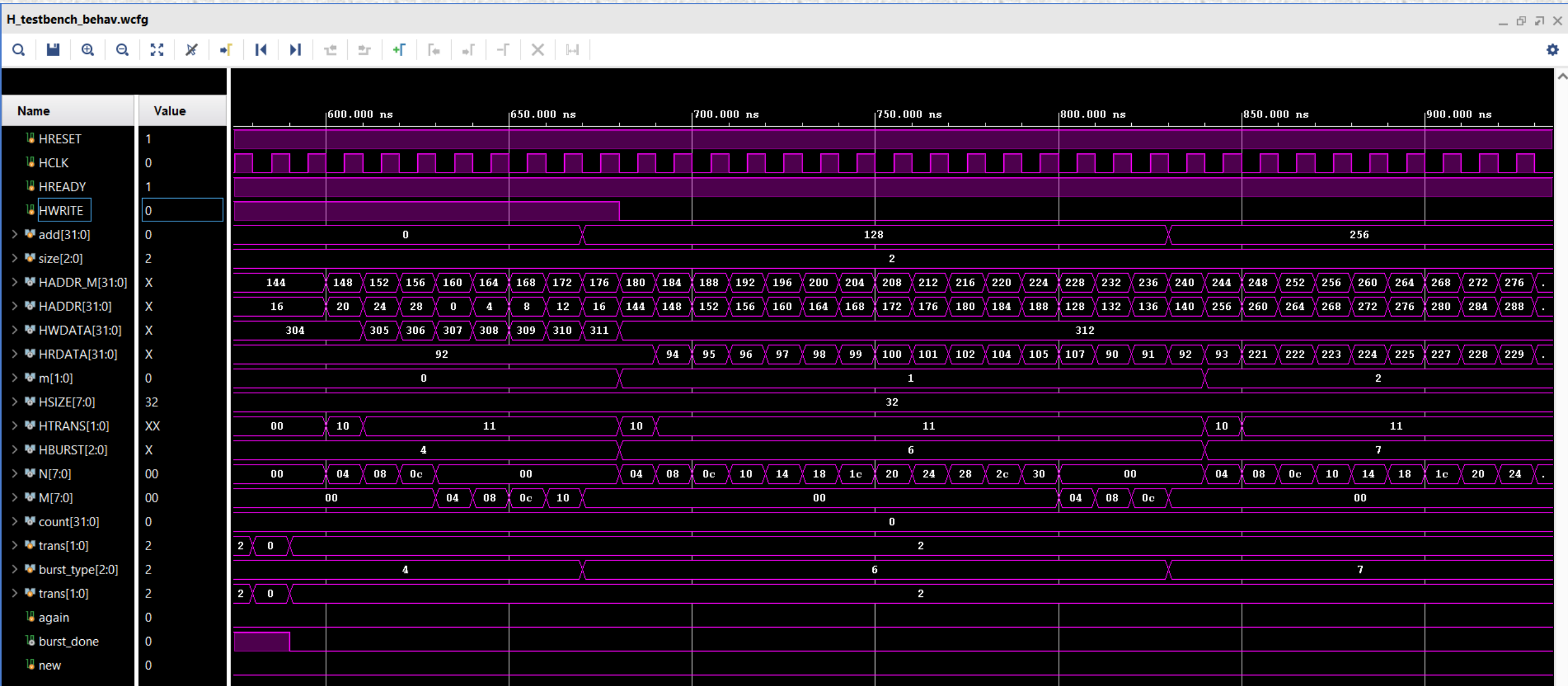
Verified output waveform:



Verified output waveform:



Verified output waveform: Moving from one slave to other slave



Elaborated Design: Schematic

